Chapter 1

Trees

Trees are one of the most versatile and widely used data structures in computer science. A *tree* is a hierarchical data structure consisting of nodes, where each node has a value and a set of references (*edges*) to its child nodes. The node at the top of the hierarchy is called the *root*, and nodes with the same parent are called *siblings*, as seen in Figure 1.1.

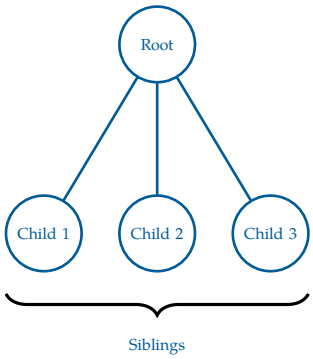


Figure 1.1: A tree with a root node and 3 children.

Trees are a subset of Graphs, a data structure comprised of nodes and edges. Graphs can be converted into trees by adding the following two constraints:

- The graph is connected, such that there exists a path between every pair of nodes
- The graph is acyclic; no cycles exist in the graph

With this comes the caveat that the number of edges in a tree is edges $= n - 1$, where n is the number of nodes in the tree. This is because a tree with n nodes must have exactly $n-1$ edges to maintain its connected and acyclic properties. If there were more than $n - 1$ edges, it would create a cycle, violating the acyclic property. If there were fewer than $n - 1$ edges, it would not be fully connected, violating the connected property.

Less formally, a *forest* is a collection of multiple trees (may be disconnected), and a *tree* is a forest with only one tree. Lots of agricultural terms in this chapter!

Trees have a unique structural property: between any two nodes, there exists exactly one simple path. This follows directly from the fact that trees are both connected and acyclic. As a result, there is only one way to move from a start node to an end node.

Although trees are often drawn with a root at the top, this is a matter of convention. Any node in a tree can be chosen as the root, and doing so induces a hierarchical structure in which edges are interpreted as parent-child relationships.

Once a root is selected, every node (except the root) has exactly one parent, and zero or more children. Because of this relationship, any randomly selected node can form a tree (we call this a *subtree*, consisting of the node and all its descendants), making the tree structure *recursive*. Moving from one node to the next is the recursive step that minimizes

the size.

Nodes with no children are called, as you'd expect, *leaves*, as they are the outermost node of a tree. The *depth* of a chosen node is the number of edges from the *root* to that node, and the *height* of a tree is the maximum depth among all nodes. Equivalent to the height of a tree is the maximum number of edges from the *root* to any leaf. You may see the *level* of a node referred to as the number of edges from the *root* to that node, similar to depth. By convention, the level and depth of the root is 0. See Figure 1.2.

Some more relevant terminology you may encounter in the context of trees:

- *Path*: A sequence of nodes where each adjacent pair is connected by an edge. This can be used to show the idea of how to get between two nodes in a tree, for example, between two cities in a flight path network.
- *Ancestor*: A node that lies on the path from the root to a given node. This is done by repeatedly traversing from a child node to a parent node until the selected node is reached.
- *Descendant*: A node that lies on the path from a given node to a leaf. This is done by repeatedly traversing from a parent node to a child node until a leaf is reached.

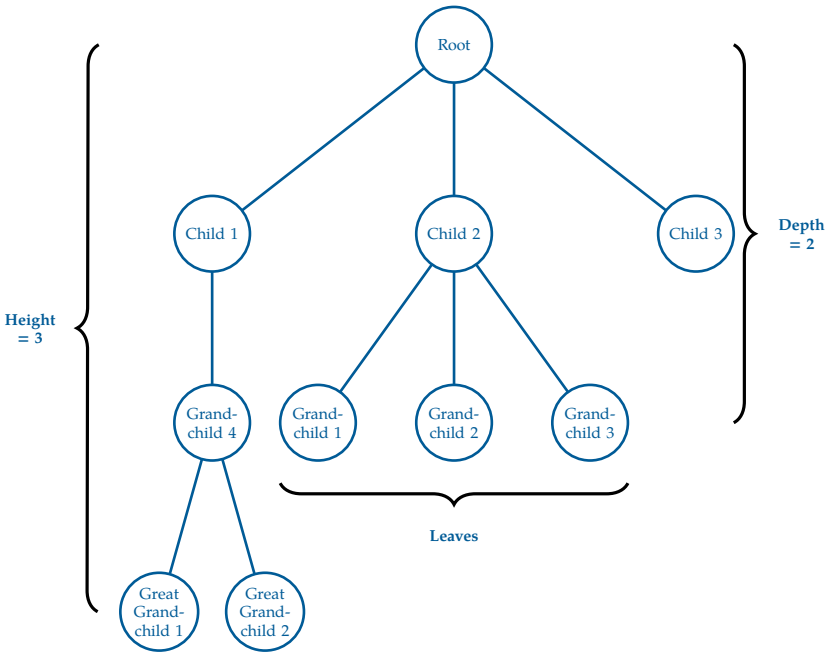


Figure 1.2: A tree with a root node and 3 children, a few grandchildren, and 2 great grandchildren.

FIXME tree proof

1.1 Binary Trees and Binary Search Trees

The most common type of tree is the *binary tree*, where each node has *at most* two children, referred to as the *left child* and the *right child*. Binary trees are widely used in various applications, including expression parsing, binary search trees, and heaps.

Let's put this practice. Recall our binary search implementation from the recursion chapter. Given a chosen word and a sorted list of words, such as a dictionary, we can find the page of the chosen word by repeatedly dividing the available pages in half and comparing

the elements on the page to the chosen word. This is much easier than, for example, going through every single page and checking if the word is on that page.

Say we wanted to represent our dictionary as a tree that we could recursively binary search through. For simplicity, we will limit our words to `mango`, `apple`, `pear`, `banana`, `anagram`, `peony` and `peach`. These 7 words form a binary tree of height 2, as seen in Figure 1.3. We select the word `mango` as the root as it is the middle word after sorting alphabetically.

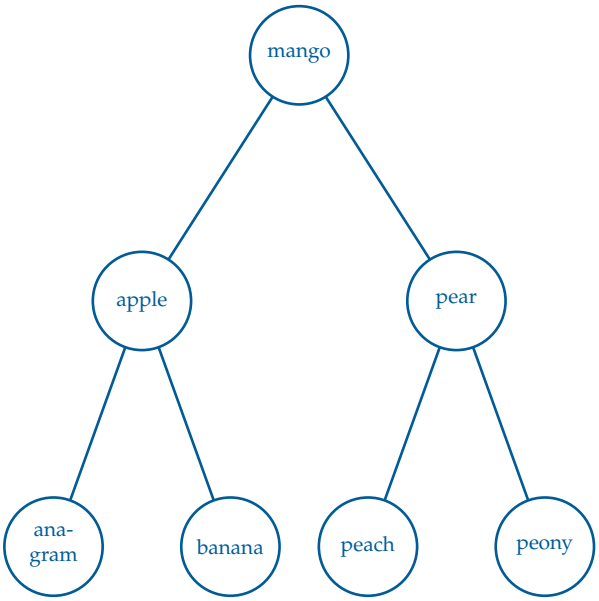


Figure 1.3: A binary search tree with 7 words.

Similar to binary search, we can find any word in the tree by starting at the root and comparing the target word to the current node's word. If the target word is less than the current node's word, we move to the left child; if it is greater, we move to the right child. We repeat this process until we find the target word or reach a leaf node, which tells us that the target word is not in the tree. This structure allows us to cut the available words in half with each step in the tree we go.

What if we want to insert into a binary search tree? We must complete four actions:

- Find the correct location for the new node.
- Create and insert the node at that location,
- Connect it to its respective parent(s).
- Ensure the BST properties are maintained.

Let's say, on our tree above, we want to add `orange` to the tree. Because of alphabetical sorting,

FIXME expand on addition, removal, and search of binary trees FIXME add exercise of a similar dictionary tree with more words, have student add and remove words, and search for words

1.2 File Structures and File System Hierarchies

Computers, since their first creation in the 1980s, have needed a way to store and organize data. Prior to computers, many systems of organization were used to store physical data, such as filing cabinets, libraries, and warehouses.

These systems often had a hierarchical structure, with a top-level category that branched out into subcategories, with these subcategories further branching out into more specific categories, and so on.

For example, a library might have two top-level categories: `fiction` and `non-fiction`. Within the category `non-fiction`, you may find `psychology`, `biographies`, `autobiographies`, `self-help`, `journalism` books, or even `nature guides`. Under the category `fiction`, you may find `romance`, `classics`, `crime`, `science fiction`, `thriller`, or `fantasy` novels. Of course, these sub-genres can have even further categorizations too.

A hospital may have patient records categorized by first letter of last name, and then sorted alphabetically from there.

With the development of the computer, sorting these types of hierarchical data became easy with the concept of directories. A *directory*, also called a *folder*, is a categorical way of organizing data (for example, by storing files or other subdirectories). We will stick to the more technical term of directories, however, any time you create a folder on your computer, you are making a *directory*. The name comes from how, in the twentieth-century, many phone books were referred to as telephone directories.

This structure naturally fits into the tree data structure:

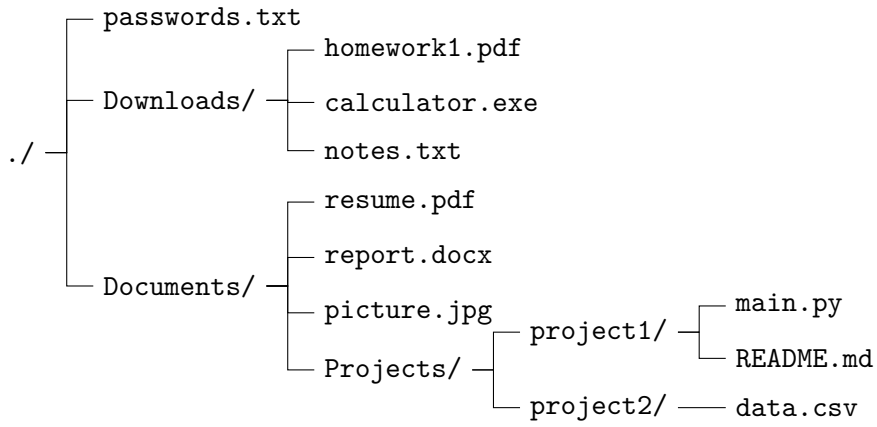
- The **root** of the tree is the top-level directory (`/` on Linux/Unix[1] systems or `C:\` on Windows).
- A **node** is naturally fits the idea of either *directories nodes* or *files nodes*.
- A **directory node** may have children, where its children are subdirectories or files.
- A **file node** is a **leaf**, meaning it has no children.

The **path** to a file describes the unique sequence of directories from the root to that file. Test again test again test again

Example

A small file system might look like:

[1] There is a very helpful video in Linux File Systems in your digital resources. Check it out!



In tree terminology:

- `./` is the root node.
- `Downloads`, `Documents`, and its subdirectories are internal nodes (directories).
- All files, such as those ending in `.txt`, `.png`, `.exe`,`.csv`, `.docx`, `.py` files are leaf nodes.

Side note: please do not store your passwords in anywhere on your computer's memory!

Exercise 1 File Paths

Working Space

In the above File Hierarchy Tree, what is the path to `data.csv`? What is the depth of the file tree at that point?

Answer on Page 7

1.3 Red-Black Trees and AVL Trees

[FIXME add content on red black trees and AVL trees]

1.4 B-Trees, Tries, and Suffix Trees

[FIXME add content on B-trees, tries, and suffix trees]

This is a draft chapter from the Kontinua Project. Please see our website (https://kontinua.org/) for more details.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 5)

The path to `data.csv` is

```
./Documents/Projects/project2/data.csv
```

The depth is the number of edges from the root to the node.

In this case, it is the number of subdirectories needed to reach `data.csv`:

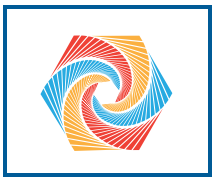
(1) `./` → `Documents`

(2) `Documents` → `Projects`

(3) `Projects` → `project2`

(4) `project2` → `data.csv`

So `data.csv` is at a depth of **4**.



Index

Ancestor, 2

binary tree, 2

depth, 2
Descendant, 2
directory, 4

edges, 1

folder, 4
forest, 1

height, 2

leaves, 2
left child, 2
level, 2

Path, 2

recursive, 1
right child, 2
root, 1

siblings, 1
subtree, 1

tree, 1